

PRACTICAL 3



CHARUSAT[®]

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY

AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms



PRACTICAL 3

Practical 3: Sorting Algorithms: Comparison-Based, Counting-Based, and Digit-Based Strategies.

Practical Aim:

To implement and compare multiple sorting algorithms and develop the ability to reason about which sorting strategy suits a given situation.

Problem 1:

A teacher has a stack of student answer sheets with marks written on them and needs to arrange them in order before entering grades. She tries three different methods: in the first, she repeatedly compares adjacent sheets and swaps them if they are out of order; in the second, she finds the lowest-marked sheet each time and places it at the front; in the third, she picks each sheet one by one and inserts it into its correct position among the already-arranged sheets. Implement all three methods and for each one, print the sorted order of marks.

Describe how each of the three methods works in your own words. If the sheets were already in order before the teacher started, which method would finish the fastest and why?

Theory:

One of the three methods can detect early that the list is already sorted and stop without doing any more work. Which method is it, how would you add that detection, and why can the other two not do the same?

Bubble Sort can detect that the list is already sorted by using a swapped variable. If no swapping occurs during a complete pass, the algorithm stops because the list is already sorted. This makes optimized Bubble Sort faster for an already sorted list, with $O(n)$ best-case time. Selection Sort cannot easily stop early because it continues searching for the minimum element in the remaining list, while basic Insertion Sort still processes each element. Therefore, optimized Bubble Sort finishes fastest when the sheets are already in order. Your code follows the adjacent comparison and swapping approach of Bubble Sort.

PRACTICAL 3

Code-Part 1(Bubble sort):

https://github.com/kanan2007s-sys/FDSA_B/tree/main/Practical3

Screenshot output:

(No input given at runtime , values were given in source code).

```
The unsorted marks of students are : 50 67 76 54 100 37 73 91 19 82
sorted marks of students :19 37 50 54 67 73 76 82 91 100
Process returned 0 (0x0)    execution time : 0.218 s
```

Description:

In this problem we need to sort (array/marks of students) using Bubble Sort. Put the biggest number at last.

Challenges:

No major challenge for selection sort. But the time complexity can be reduced told by sir that the only wrong thing.

PRACTICAL 3

Part 2(Selection sort):

Code:

https://github.com/kanan2007s-sys/FDSA_B/tree/main/Practical3

Output Screenshot:

```
int main() {  
    int ar[10]={11,12,34,43,23,22,55,66,44,100};  
    int temp=0;
```



```
11 12 22 23 34 43 44 55 66 100  
Process returned 0 (0x0) ... execution time: 0.001 s
```

Description:

Here, we need to do selection sort. Selection Sort divides the list into sorted and unsorted parts. It finds the smallest mark from the unsorted part and places it at the beginning of that part. This is repeated until all marks are in ascending order.

Challenge:

In this practical I was just confused if the implementation was right type of sorting method.

PRACTICAL 3

Part 3(Insertion sort):

Code:

https://github.com/kanan2007s-sys/FDSA_B/tree/main/Practical3

Output Screenshot:

```
int marks[5] = {70, 50, 90, 60, 80};  
int key;
```

```
Sorted marks: 50 60 70 80 90  
Process returned 0 (0x0)    execution time : 0.079 s
```

Description:

Here , we used insertion sort. In this we put the values in sorted array. First we take portion of array which is array or element , then compare every element and place it at required place.

Challenge:

How to use iterative part to sort elements in Insertion sort was very difficult after learning the concept in class I was able to do it.

Github Repository link:

https://github.com/kanan2007s-sys/FDSA_B/tree/main/Practical3

Conclusion:

The three sorting algorithms—Bubble Sort, Selection Sort, and Insertion Sort—were implemented and compared. Each algorithm uses a different approach to arrange the elements, and optimized Bubble Sort can detect an already sorted array and stop early, making it efficient in the best case.

PRACTICAL 3

Problem 2:

A paint shop has buckets labeled with one of three colour codes — 0, 1, or 2 — but they are stored in a random order. The shop owner wants all 0s together, then all 1s, then all 2s, without using any extra storage. Given the list of colour codes, rearrange them in place and print the result.

Describe the approach you used. Does your solution make more than one pass through the list? Can you think of a way to do it in exactly one pass?

Theory:

If your solution makes two passes through the list, can you identify which decision in your logic forces the second pass? Is there a way to restructure that decision to eliminate it?

If the solution uses counting, it requires two passes: the first pass counts the number of 0s, 1s and 2s, and the second pass places them into the array. The decision to count first and rearrange later causes the second pass. This can be eliminated using the Dutch National Flag algorithm, which uses three pointers (low, mid, and high) to rearrange 0s, 1s and 2s in a single pass. It works in $O(n)$ time and $O(1)$ extra space.

Code:

https://github.com/kanan2007s-sys/FDSA_B/tree/main/Practical3

Output Screenshot;

```
int ar[10]={0,1,1,1,2,2,1,0,0,2};
```

So the final array with Selection sort is: 0 0 0 1 1 1 2 2 2

Description:

Here , we used selection sort since it was not specified to use which sorting method.

PRACTICAL 3

Challenge:

I could not find which sorting method will be best for the given question but I used selection sort as it also satisfies the condition to not take extra variable.

Github Repository link:

https://github.com/kanan2007s-sys/FDSA_B/tree/main

Conclusion:

The colour codes 0, 1, and 2 were rearranged in-place without using extra storage. The Dutch National Flag approach can solve the problem in a single pass using three pointers, making it efficient in both time and space.